CSE Department – Faculty of Engineering - MSA

Spring 2026

CSE264 Operating System

Course Instructor: Assoc. Professor Mokhtar A. A. Mohamed.

Project Title: CPU Scheduling

Student Name	**Ahmed Hossam Mohamed Ezz-Eldin Mohamed.**	**Student ID**	**248141.**
Student Name	**Ahmed Amr Ahmed Ismail Jabr.**	**Student ID**	**241659.**
Student Name	**Mohamed Amr Mohamed Salama Mohamed Eissa.**	**Student ID**	**243835.**
TA Name	**Eng. Hussien Mostafa**	**Grade:**	**/ 10**

GitHub Link For That Project:
https://github.com/Ahmed-Hossam2004/CPU-Scheduler-OSProject

1. Responsibilities:

Member Name	Responsibilities
Ahmed Hossam.	Create Classes : • SchedulerController. • ProcessLauncher.
Ahmed Amr.	Create Classes : • SchedulingAlgorithms. • WorkerProcess.
Mohamed Amr.	Create Classes : • ProcessControlBlock • TelemetryReporter.

2. Introduction:

In modern operating systems, the CPU Scheduler is the core component responsible for managing how processes share limited processor time. It determines the order and duration of execution for various tasks, aiming to maximize system throughput, minimize waiting time, and ensure fairness among competing applications.

This project implements a comprehensive CPU Scheduler Simulator developed specifically for the Ubuntu Linux environment. Unlike traditional simulators that only calculate theoretical numbers, this system bridges the gap between simulation and reality by controlling actual child processes within the host operating system.

- **Objectives**
  - Process Management: Dynamically spawning and terminating real worker processes using Java's ProcessBuilder.

- Kernel Signaling: Implementing process state control (Ready vs. Running) by sending SIGSTOP and SIGCONT signals to the Linux kernel via the kill command.
- Algorithm Analysis: Providing a comparative study of three fundamental scheduling strategies:
    - **First-Come, First-Served (FCFS):** A non-preemptive approach prioritizing arrival order.
    - **Round Robin (RR):** A preemptive, time-sharing strategy using a user-defined quantum.
    - **Priority Scheduling:** An importance-based execution model.
- Visual Telemetry: Generating real-time Gantt Charts and performance reports that calculate Turnaround Time (TAT) and Waiting Time (WT) based on a deterministic logical cycle clock.

3. Short Description of the Design:

The system is designed as a Multithreaded CPU Scheduler Simulator that bridges the gap between high-level Java application logic and low-level Linux Process Management. The design follows a modular Model-View-Controller (MVC) pattern to ensure a clean separation between the user interface and the scheduling engine.

- **Key Design Pillars:**
    - Hybrid Simulation Model: The design uses a Logical Cycle Clock for deterministic scheduling calculations (TAT/WT) while simultaneously managing Real Ubuntu

Processes in the background. This ensures that the simulation is visually accurate without being affected by system background noise.

- Linux Kernel Integration: The design leverages the ProcessBuilder API to spawn child processes and utilizes the /usr/bin/kill utility. By sending SIGSTOP and SIGCONT signals, the scheduler manually forces processes into Ready or Running states, mimicking a real OS kernel.
- State-Driven PCB: At the heart of the design is the Process Control Block (PCB), which acts as the primary data structure. It stores critical metadata, including Arrival Cycles, Burst Times, Priorities, and the actual OS Process ID (osPid).
- Dynamic Algorithm Engine: The scheduler is designed with a switchable logic core, allowing for a comparative analysis of FCFS, Round Robin (preemptive), and Priority scheduling using a shared Ready Queue.
- Visual Telemetry: The UI component features a custom-painted Gantt Chart Panel that dynamically scales to represent the execution timeline, providing a clear visual representation of context switching and process lifecycle.

4. Explanation of Implementation:

The implementation of this project focuses on the synchronization between a Java-based simulation and the Ubuntu Linux Kernel. It is structured into three distinct layers: the Process Management Layer, the Scheduling Logic, and the Data Reporting Layer.

1. Process Management & Linux Integration

The implementation starts by spawning a real OS process for every task added by the user.

- **Worker Spawning:** The ProcessLauncher uses ProcessBuilder to execute a secondary JVM running WorkerProcess.java.
- **Immediate Preemption:** To simulate the Ready State, the launcher immediately sends a SIGSTOP signal using the Linux kill command. This ensures the process is "paused" by the kernel and does not consume CPU until the scheduler explicitly allows it.
- **Context Switching:** During simulation, the SchedulingAlgorithms use SIGCONT (Continue) to wake the process and SIGSTOP to pause it again once its logical cycle ends.

2. The Logical Cycle Clock

To ensure the simulation results are stable and not affected by background computer lag, the implementation uses Logical Time (Cycles) rather than system milliseconds.

- **Discrete Ticks:** Each "burst" in the simulation is a discrete integer.
- **Deterministic Math:** Calculations for Turnaround Time (TAT) and Waiting Time (WT) use these fixed cycle integers, ensuring the Gantt Chart and the Performance Table are perfectly synchronized.

3. Scheduling Algorithm Implementation

Each algorithm is implemented using standard Java Collection structures:

- **FCFS:** Implemented by sorting the Ready Queue by the arrivalCycle property.
- **Priority:** Uses a multi-level sort (Priority first, then Arrival as a tie-breaker).

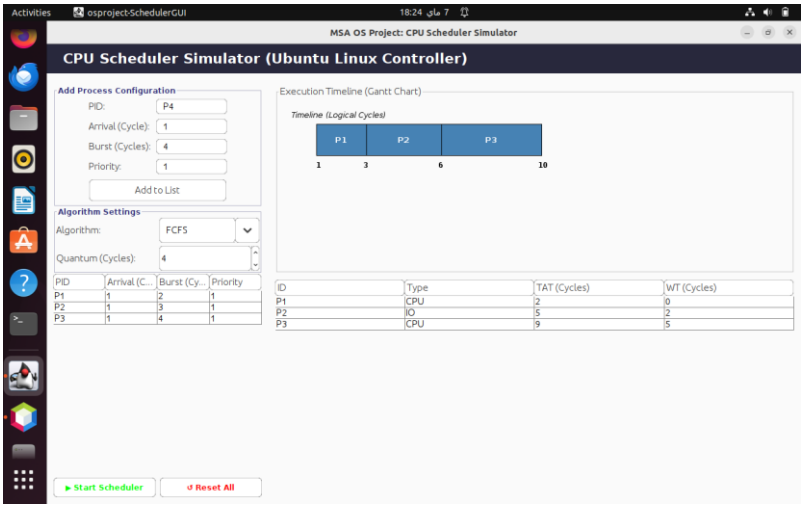
- **Round Robin:** Implemented using a LinkedList as a circular queue. The logic checks if remainingTime > quantum; if so, it executes a burst equal to the quantum and moves the process to the back of the queue.

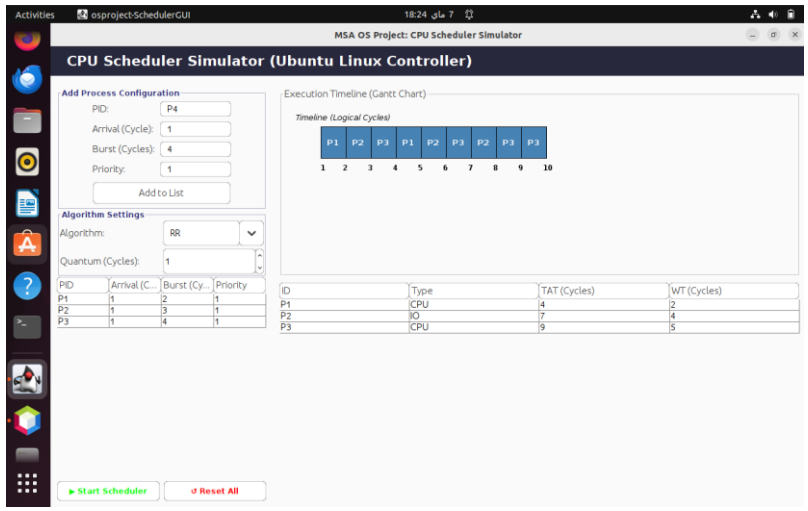
4. UI and Visualization

The interface is implemented using Java Swing.

- **GanttChartPanel:** A custom component that overrides paintComponent to draw rectangles representing process execution. It uses a SCALE constant to map logical cycles to pixels.
- **Telemetry Reporting:** Upon completion, the TelemtryReporter iterates through the completed PCB objects to calculate averages and print a formatted summary to the console using System.out.printf.

5. Screenshots of Execution and Outputs:





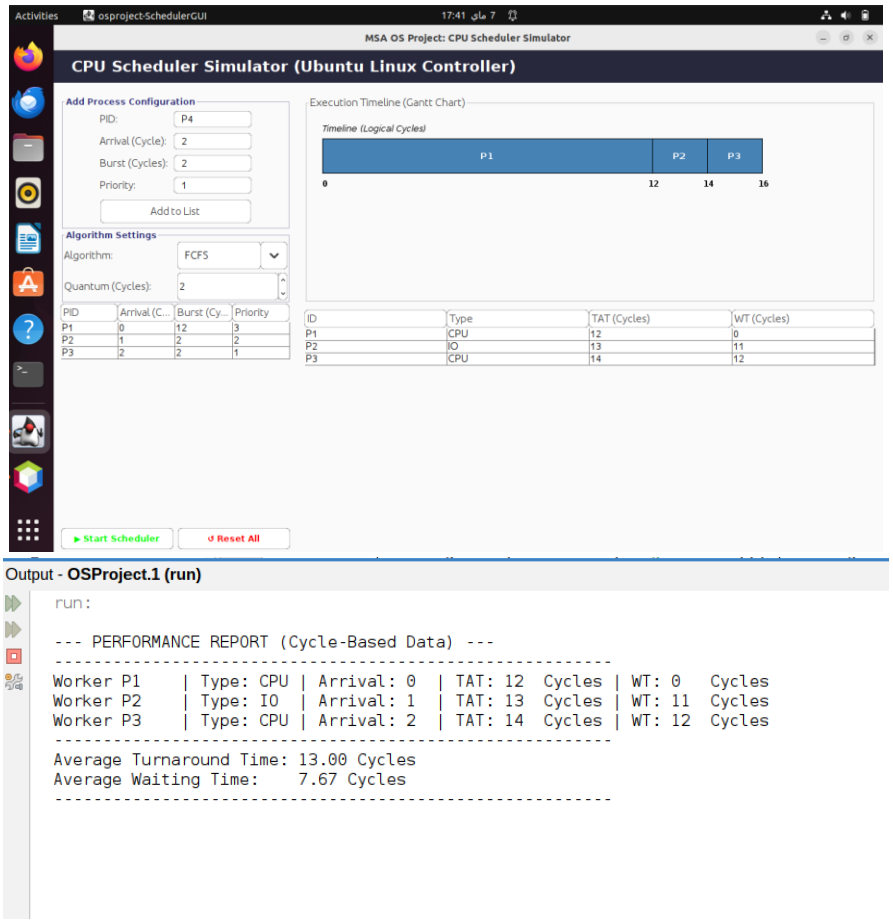

```
--- PERFORMANCE REPORT (Cycle-Based Data) ---
-------------------------------------------------------
Worker P1    | Type: CPU | Arrival: 1  | TAT: 2   Cycles | WT: 0   Cycles
Worker P2    | Type: IO  | Arrival: 1  | TAT: 5   Cycles | WT: 2   Cycles
Worker P3    | Type: CPU | Arrival: 1  | TAT: 9   Cycles | WT: 5   Cycles
-------------------------------------------------------
Average Turnaround Time: 5.33 Cycles
Average Waiting Time:    2.33 Cycles
-------------------------------------------------------
```

```
--- PERFORMANCE REPORT (Cycle-Based Data) ---
-------------------------------------------------------
Worker P1    | Type: CPU | Arrival: 1  | TAT: 4   Cycles | WT: 2   Cycles
Worker P2    | Type: IO  | Arrival: 1  | TAT: 7   Cycles | WT: 4   Cycles
Worker P3    | Type: CPU | Arrival: 1  | TAT: 9   Cycles | WT: 5   Cycles
-------------------------------------------------------
Average Turnaround Time: 6.67 Cycles
Average Waiting Time:    3.67 Cycles
-------------------------------------------------------
```

6. Test Cases and Results:

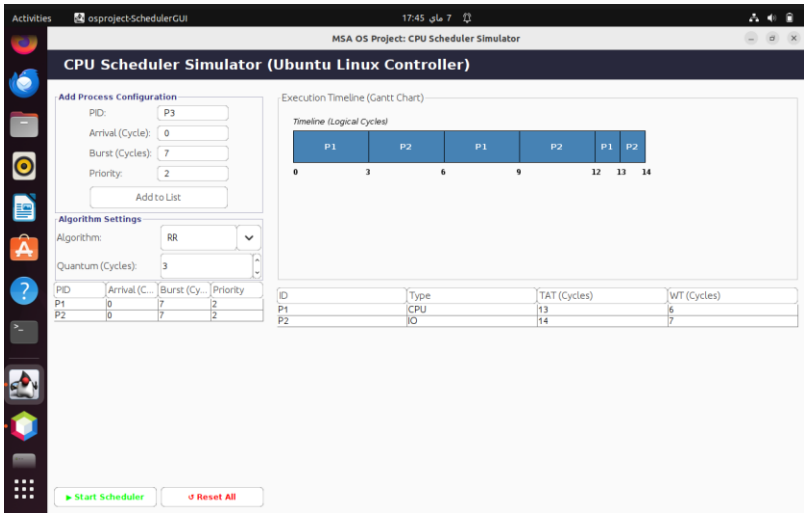
- **Test Case 1: The "Convoy Effect"**
- **Goal:** Prove that in **FCFS**, a single long process arriving first ruins the waiting time for all subsequent short processes.

Process ID	Arrival Cycle	Burst Time	Priority	Task Type
P1	0	12	3	CPU
P2	1	2	2	IO
P3	2	2	1	CPU



- **Test Case 2: Round Robin Preemption & Quantum**
- **Goal:** Verify that the SIGSTOP and SIGCONT logic correctly handles a process that is interrupted before it finishes
- **Settings:** Set **Quantum = 3**.

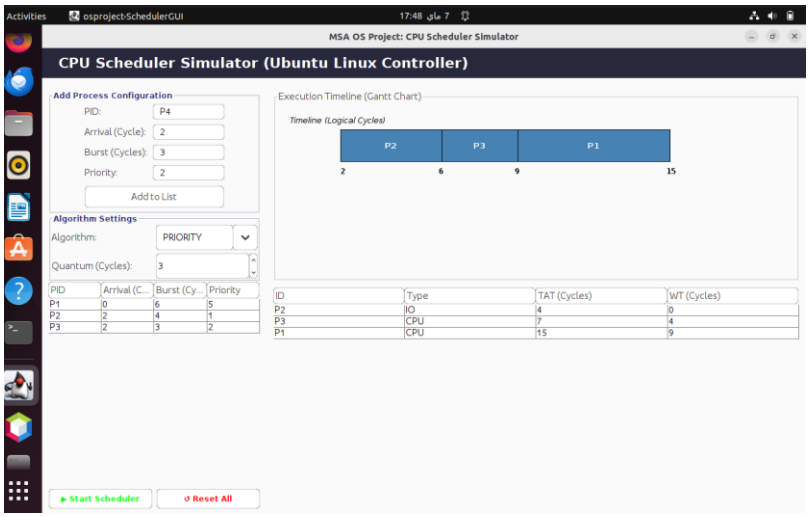
Process ID	Arrival Cycle	Burst Time	Priority	Task Type
P1	0	7	2	CPU
P2	0	7	2	IO




```
--- PERFORMANCE REPORT (Cycle-Based Data) ---
-------------------------------------------------------
Worker P1    | Type: CPU | Arrival: 0  | TAT: 13  Cycles | WT: 6   Cycles
Worker P2    | Type: IO  | Arrival: 0  | TAT: 14  Cycles | WT: 7   Cycles
-------------------------------------------------------
Average Turnaround Time: 13.50 Cycles
Average Waiting Time:    6.50 Cycles
-------------------------------------------------------
```

- **Test Case 3: Priority with Late Arrival**
- **Goal:** Test if your Comparator correctly prioritizes importance over arrival when multiple processes are waiting.

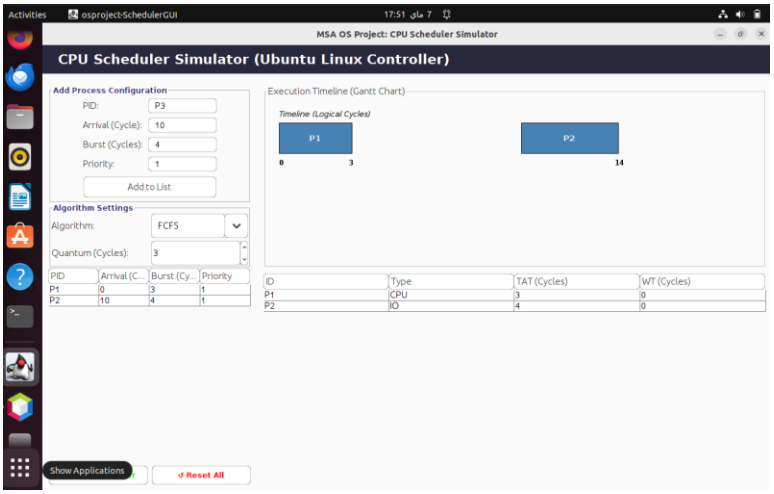
Process ID	Arrival Cycle	Burst Time	Priority	Task Type
P1	0	6	5 (Low)	CPU
P2	2	4	1 (High)	IO
P3	2	3	2 (Med)	CPU




```
--- PERFORMANCE REPORT (Cycle-Based Data) ---
-------------------------------------------------------
Worker P2    | Type: IO  | Arrival: 2  | TAT: 4   Cycles | WT: 0   Cycles
Worker P3    | Type: CPU | Arrival: 2  | TAT: 7   Cycles | WT: 4   Cycles
Worker P1    | Type: CPU | Arrival: 0  | TAT: 15  Cycles | WT: 9   Cycles
-------------------------------------------------------
Average Turnaround Time: 8.67 Cycles
Average Waiting Time:    4.33 Cycles
-------------------------------------------------------
```

- **Test Case 4: CPU Idle Time (The Gap)**
- **Goal:** Ensure the scheduler handles a situation where the CPU has nothing to do for a period.

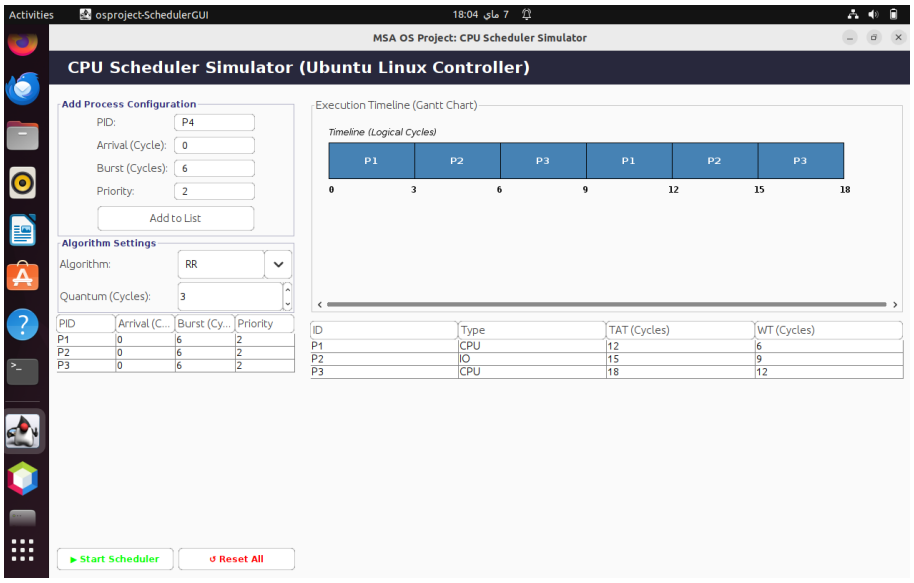
Process ID	Arrival Cycle	Burst Time	Priority	Task Type
P1	0	3	1	CPU
P2	10	4	1	IO




```
--- PERFORMANCE REPORT (Cycle-Based Data) ---
-------------------------------------------------------
Worker P1    | Type: CPU | Arrival: 0  | TAT: 3   Cycles | WT: 0   Cycles
Worker P2    | Type: IO  | Arrival: 10 | TAT: 4   Cycles | WT: 0   Cycles
-------------------------------------------------------
Average Turnaround Time: 3.50 Cycles
Average Waiting Time:    0.00 Cycles
-------------------------------------------------------
```

- **Test Case 5: The "Perfect Fit" Quantum (Focus: RR)**
- **Goal:** Observe the efficiency of the Round Robin queue when Burst Times are exact multiples of the Quantum. This ensures the scheduler handles the transition between turns without creating "empty" cycles.
- **Settings:** Set **Quantum = 3**.

Process ID	Arrival Cycle	Burst Time	Priority	Task Type
P1	0	6	2	CPU
P2	0	6	2	IO
P3	0	6	2	CPU

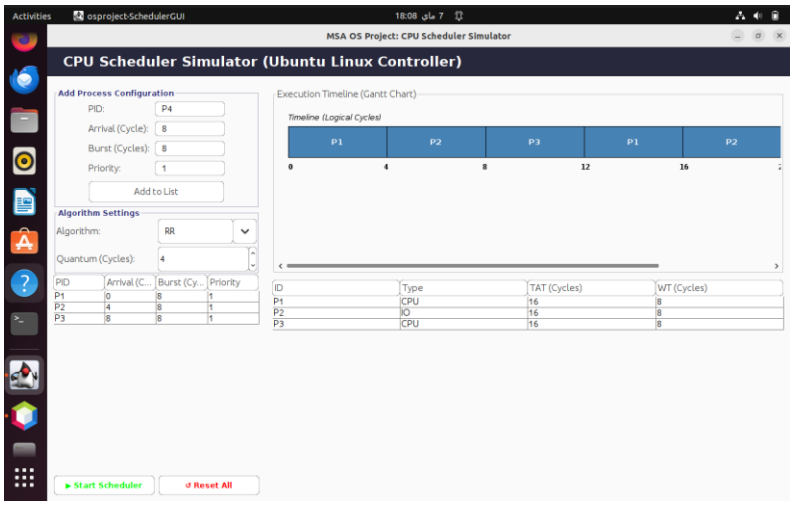



```
--- PERFORMANCE REPORT (Cycle-Based Data) ---
-------------------------------------------------------
Worker P1    | Type: CPU | Arrival: 0  | TAT: 12  Cycles | WT: 6   Cycles
Worker P2    | Type: IO  | Arrival: 0  | TAT: 15  Cycles | WT: 9   Cycles
Worker P3    | Type: CPU | Arrival: 0  | TAT: 18  Cycles | WT: 12  Cycles
-------------------------------------------------------
Average Turnaround Time: 15.00 Cycles
Average Waiting Time:    9.00 Cycles
-------------------------------------------------------
```

- **Test Case 6: Staggered "Heavy" Arrivals (Focus: FCFS vs RR)**
- **Goal:** Test how the scheduler handles a "staircase" arrival pattern, where a new, heavy process enters the system exactly when the current one is halfway through its execution.
- **Settings for RR:** Set **Quantum = 4.**

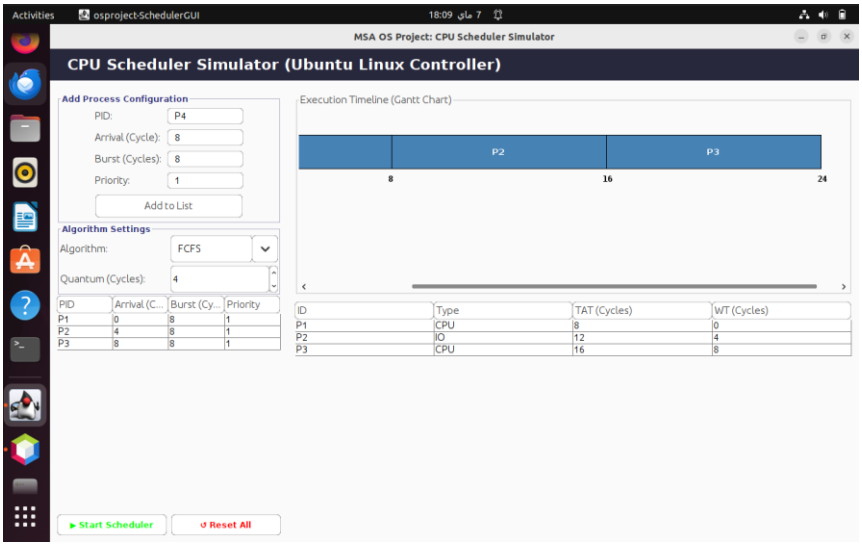
Process ID	Arrival Cycle	Burst Time	Priority	Task Type
P1	0	8	1	CPU
P2	4	8	1	IO
P3	8	8	1	CPU

RR:




```
--- PERFORMANCE REPORT (Cycle-Based Data) ---
-------------------------------------------------------
Worker P1    | Type: CPU | Arrival: 0  | TAT: 16  Cycles | WT: 8   Cycles
Worker P2    | Type: IO  | Arrival: 4  | TAT: 16  Cycles | WT: 8   Cycles
Worker P3    | Type: CPU | Arrival: 8  | TAT: 16  Cycles | WT: 8   Cycles
-------------------------------------------------------
Average Turnaround Time: 16.00 Cycles
Average Waiting Time:    8.00 Cycles
-------------------------------------------------------
```

FCFS:



```
--- PERFORMANCE REPORT (Cycle-Based Data) ---
-------------------------------------------------------
Worker P1    | Type: CPU | Arrival: 0  | TAT: 8   Cycles | WT: 0   Cycles
Worker P2    | Type: IO  | Arrival: 4  | TAT: 12  Cycles | WT: 4   Cycles
Worker P3    | Type: CPU | Arrival: 8  | TAT: 16  Cycles | WT: 8   Cycles
-------------------------------------------------------
Average Turnaround Time: 12.00 Cycles
Average Waiting Time:    4.00 Cycles
-------------------------------------------------------
```

7. Conclusion:

The development of this CPU Scheduler Simulator demonstrates the critical intersection between theoretical scheduling logic and practical process control within the Ubuntu Linux environment. By successfully integrating Java's process management capabilities with low-level Linux kernel signals, the project provides a transparent look at how an operating system manages competing workloads. Through the comparative analysis of FCFS, Round Robin, and Priority algorithms, the following **key insights were observed:**

- **Predictability vs. Fairness:** While FCFS offers simplicity and predictability, Round Robin proved to be more effective at ensuring fairness and preventing process starvation through the use of a dynamic time quantum.
- **Logical Synchronization:** The implementation of a **Logical Cycle Clock** was essential for creating a deterministic environment, ensuring that the visual Gantt Chart and performance telemetry remained accurate regardless of system hardware fluctuations.
- **Resource Management:** The use of SIGSTOP and SIGCONT highlights the importance of the **Ready** and **Waiting** states in the process lifecycle, proving that a scheduler's primary role is the efficient conservation and allocation of CPU resources.

Ultimately, this project serves as a robust educational tool, transforming abstract scheduling formulas into a tangible, controllable system. It reinforces the core principles of Operating Systems—efficiency, fairness, and resource optimization—while providing a foundation for more advanced explorations into multi-core scheduling and real-time process management.

8. References:

- **Oracle Corporation. (2026).** *Java platform, standard edition (Java SE) documentation - ProcessBuilder class*. https://docs.oracle.com/en/java/javase/21/docs/api/java.base/java/lang/ProcessBuilder.html
- **Silberschatz, A., Galvin, P. B., & Gagne, G. (2018).** *Operating system concepts* (10th ed.). Wiley.
- **The IEEE and The Open Group. (2018).** *The Open Group base specifications issue 7, 2018 edition - kill utility*. https://pubs.opengroup.org/onlinepubs/9699919799/utilities/kill.html